

به نام خدا

گزارش تمرین کامپیوتری ششم

سیگنال‌ها و سیستم‌ها-دکتر اخوان

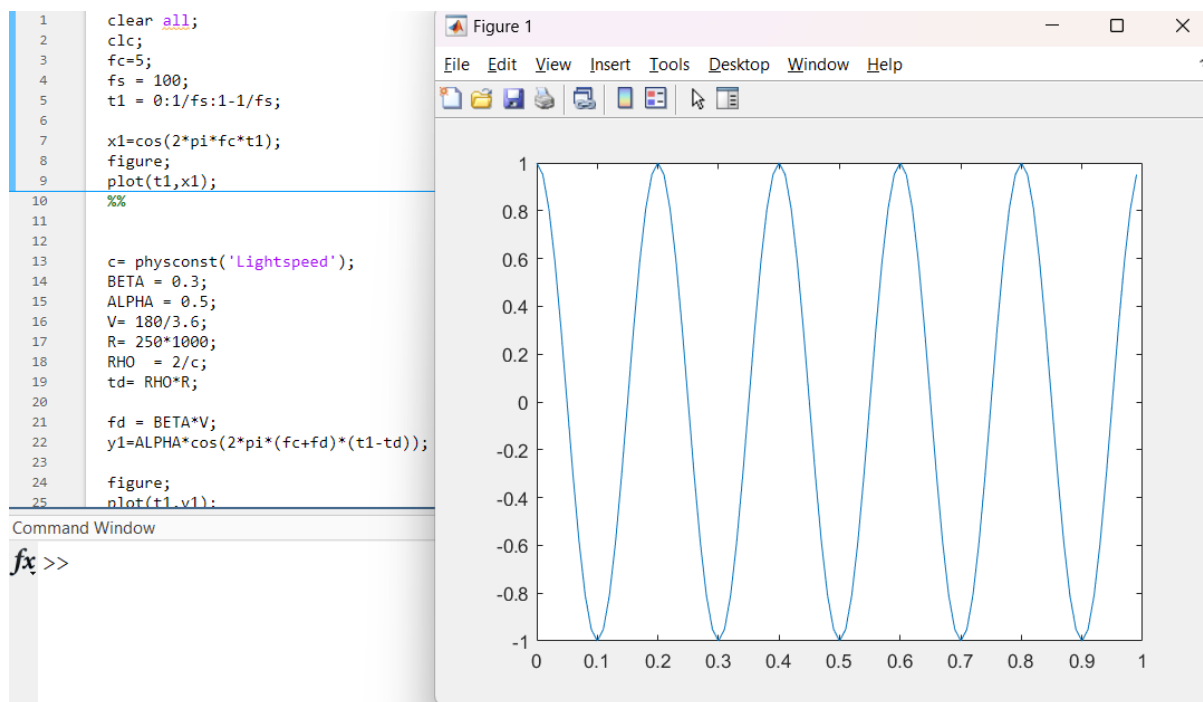
امیرمحمد میرحسینی ۸۱۰۱۰۲۶۰۱

بخش اول)

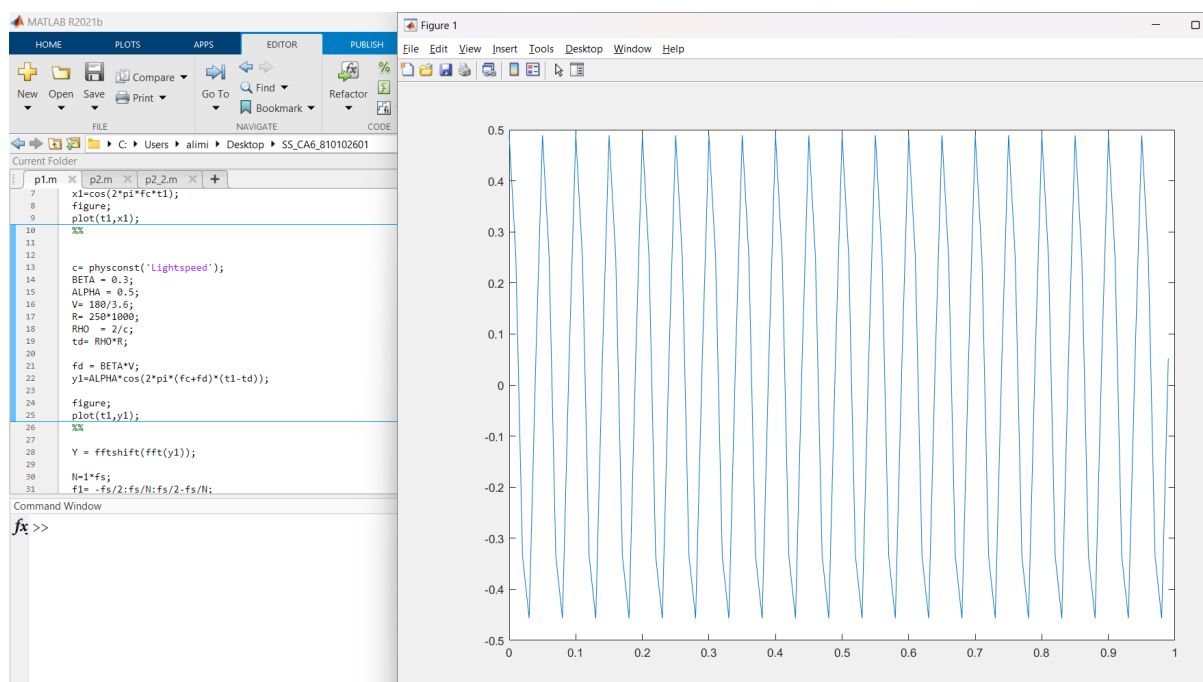
لازم به ذکر است مقادیر نهایی فاصله و سرعت در این بخش که در متلب اسکرینشات گرفته شده اند به ترتیب متر و متر بر ثانیه می باشند.

File : p1.m

تمرین ۱-۱)



تمرین ۲-۱)



```

26 %%
27
28 Y = fftshift(fft(y1));
29
30 N=1*fs;
31 f1= -fs/2:fs/N:fs/2-fs/N;
32 Y = Y/max(abs(Y));
33 figure;
34 plot(f1,abs(Y));
35 %
36 figure ;
37 plot(f1,angle(Y));
38 Yfsize = length(Y);
39 half_idx = (floor(Yfsize/2)+2):Yfsize;
40 Z=Y(half_idx);
41 idx = find(Z > 0.1 ) ;
42
43 f_new = idx;
44 phi_new = angle(Z(idx));
45 f_d_find = f_new - fc;
46
47 t_d_find = phi_new / (-2*pi*(f_d_find+fc));
48

```

با توجه به راهنمایی و تبدیل فوریه آن همانطور که مشاهده می‌شود تبدیل فوریه تابع فقط در دو نقطه مقدار دارد. که این دو نقطه دارای فرکانس‌های قرینه هم هستند. به همین دلیل ما سیگنال را به فضای فوریه برده و با $fftshift$ فرکانس صفر را به مرکز آرایه منتقل می‌شود. به همین دلیل چون فرکانس‌ها قرینه هم هستند. از نصف تا آخر فوریه یافته سیگنال را در نظر می‌گیریم که ایندکس ماکزیمم همان f_{new} شود. پس از استخراج f_{new} منهای f_c می‌کنیم و f_d پیدا می‌شود. Phi_new هم که مقدار فاز در ایندکس ماکزیمم (همان فرکانس نیو) می‌باشد. سپس به کمک f_{new} ، t_d را بدست می‌آوریم.

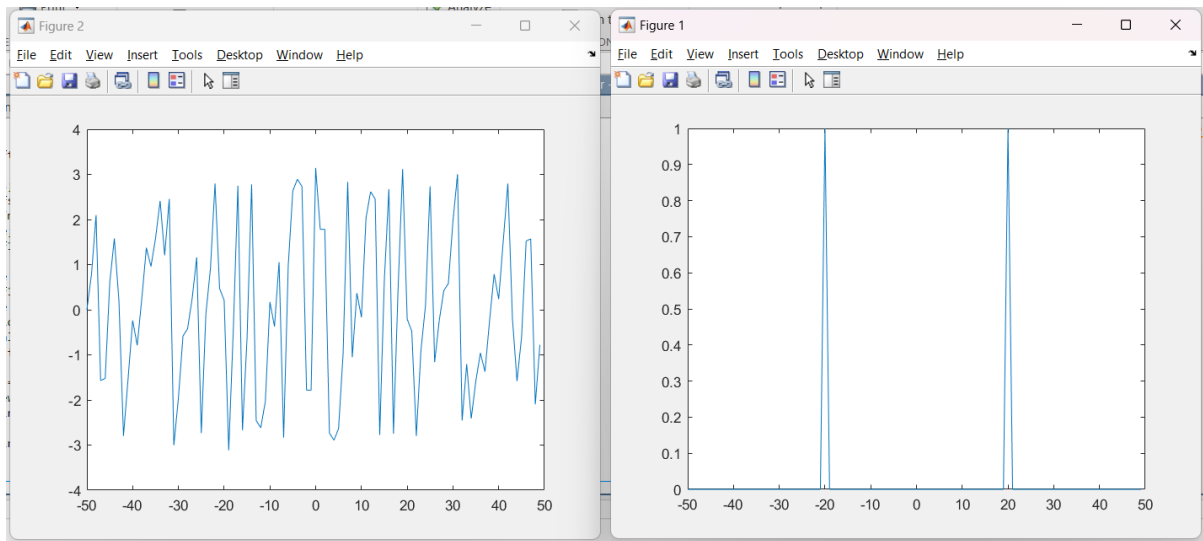
$$t_{new} = t + \phi_{new} / (-2\pi(f_d + f_c))$$

محاسبه می‌شود

که برابر با مقدار اولیه هستند.

t_d_find	0.0017	f_d_find	15
td	0.0017	f_new	20
phi_new	-0.2096	fd	15

در زیر نمودار چپ فاز و نمودار سمت راست اندازه فوریه یافته سیگنال دریافتی می‌باشد.



تمرین ۱-۴)

```
%%
flag_t_d = 0;
flag_f_d = 0;
for i = 0.001:0.02:100

    noisedy1 = y1 + 0.01*i*(randn(1,length(y1)));

    NoisedY = fftshift(fft(noisedy1));
    NoisedY=NoisedY/max(abs(NoisedY));
    noisedZ=NoisedY(half_idx);
    [maxVal , jdx] = max(noisedZ);
    f_new_noised = jdx;
    phi_new_noised = angle(noisedZ(jdx));
    f_d_find_noised = f_new_noised - fc;

    t_d_find_noised = phi_new_noised / (-2*pi*(f_d_find_noised+fc));

    if t_d_find_noised ~= t_d_find && flag_t_d == 0
        fprintf('wrong finded t_d for i= %d is: %f\n: ', i,t_d_find_noised);
        fprintf('wrong finded R is: %f km\n: ', t_d_find_noised/(1000*RHO));

        flag_t_d = 1;|
    end

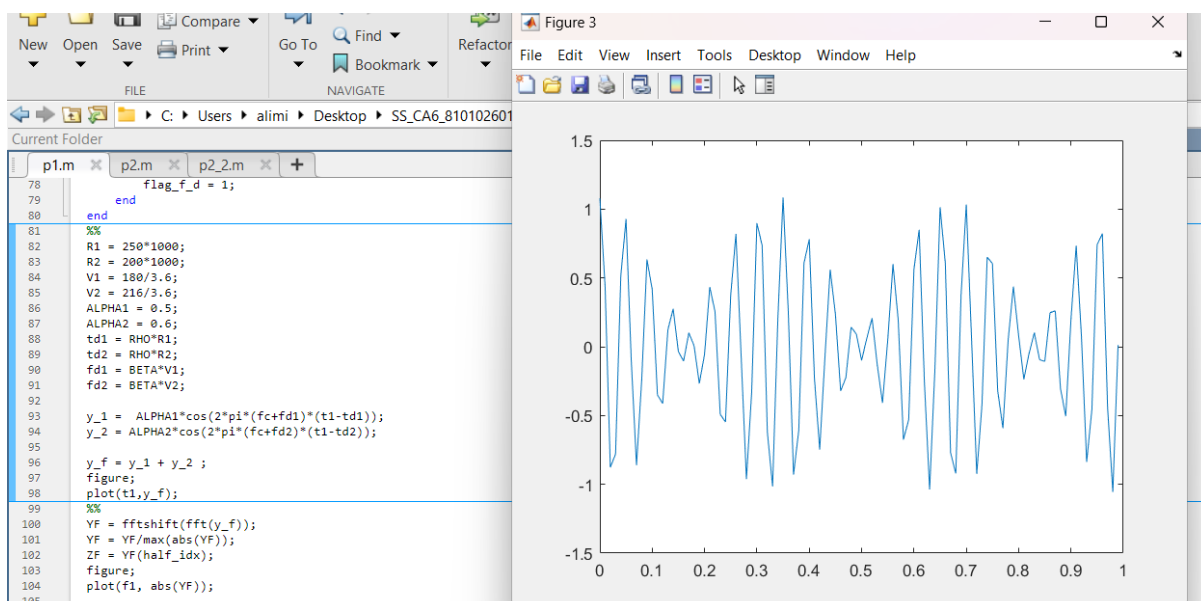
    if f_d_find_noised ~= f_d_find && flag_f_d == 0
        fprintf('wrong finded f_d for i= %d is: %f\n: ', i,f_d_find_noised);
        fprintf('wrong finded V is: %f km/h\n: ', f_d_find_noised/BETA*3.6);
        flag_f_d = 1;
    end
end
%%
```

$$\sigma = 0.01 * i$$

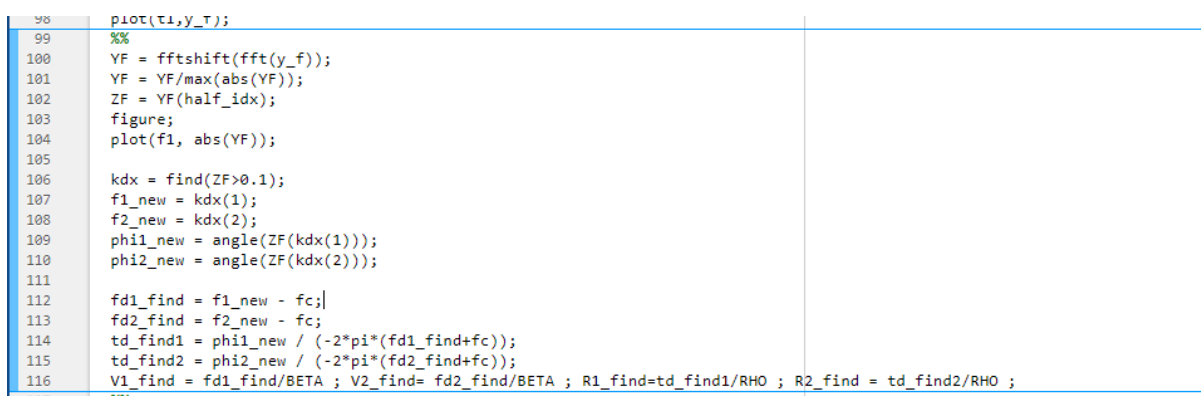
سیگما در بالا انحراف معیار نویز افزوده شده به سیگنال است. همانطور که مشاهده می‌کنید با کمترین نویز داده شده (اینجا با انحراف معیار یک ده هزارم!) t_{delay} به سرعت دچار خرابی می‌شود. بنابراین t_{delay} بیشترین حساسیت را نسبت به نویز دارد. پس از آن f_d در انحراف معیار حدودا ۰.۶۵ حدودا دچار خرابی می‌شود و فرکانس داپلر را برای سیگنال دریافتی بخش قبل را اشتباه اندازه می‌گیرد.

```
: wrong finded t_d for i= 1.000000e-03 is: 0.001668
: wrong finded R is: 250.001351 km
: wrong finded f_d for i= 6.694100e+01 is: 42.000000
: wrong finded V is: 504.000000 km/h
```

تمرین ۱-۵)



تمرین ۱-۶)

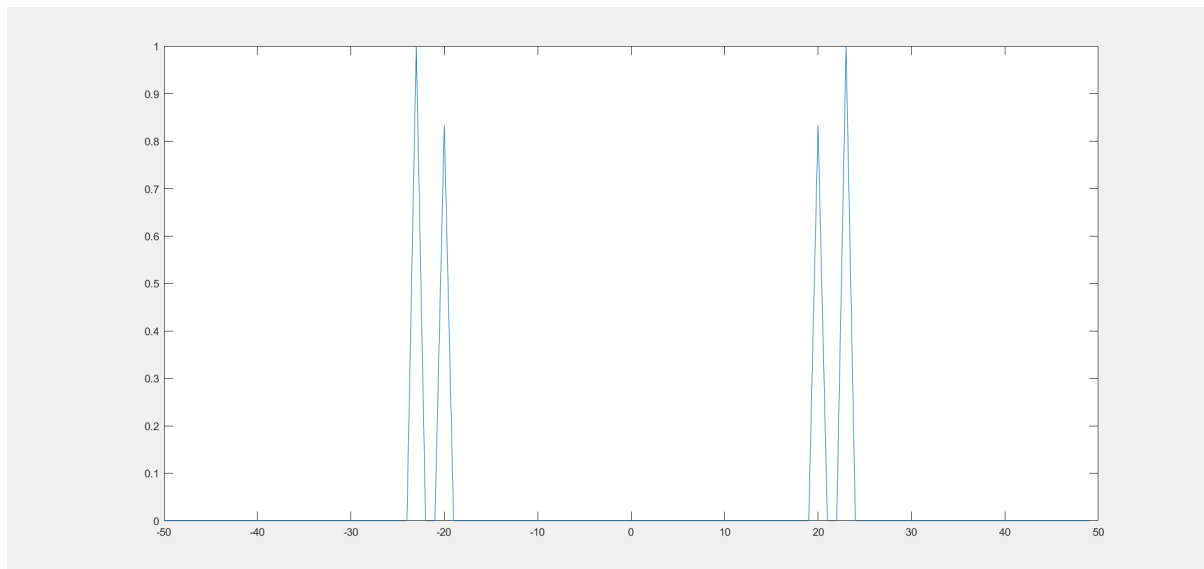


همانند بخش قبل می‌باشد با این تفاوت که بجای دو تا پیک ، ۴ تا پیک در حوزه فوریه داریم چون که دو تا سیگنال سینوسی با فرکانس متفاوت را دریافت می‌کنیم. به همین ترتیب مانند قبل با *fftshift* فرکانس صفر را به مرکز ارایه برده و از نصف به بعد را حساب می‌کنیم و ایندکس‌های ماکزیمم همان فرکانس‌های f_{new} ها می‌شود. و ϕ_{new} (فاز هریک در همان فرکانس خودش می‌باشد). ها را نیز با فاز همان ایندکس‌های فرکانس ماکزیمم اندازه می‌گیریم. و به این ترتیب ، $R1$ ، $R2$ ، $V1$ ، $V2$ را اندازه می‌گیریم.

R1	250000	8 1x1
R1_find	2.5000e+...	8 1x1
R2	200000	8 1x1
R2 find	2.0000e+...	8 1x1

td_find1	0.0017			
td_find2	0.0013			
V	50			
V1	50	fd1	15	8 1x1
V1_find	50	fd1_find	15	8 1x1
V2	60	fd2	18	8 1x1
V2_find	60	fd2 find	18	8 1x1

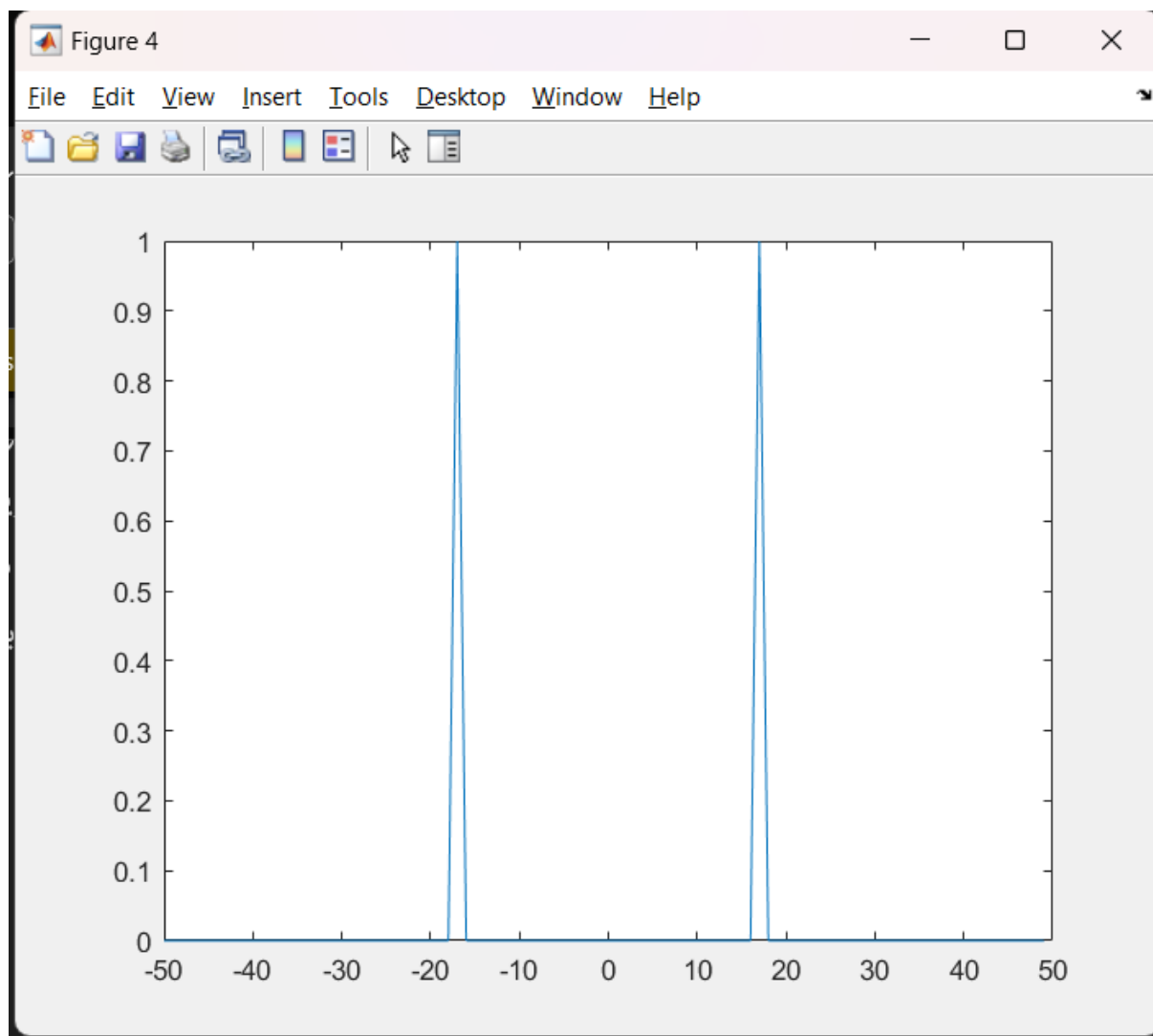
در زیر نمودار اندازه تبدیل فوریه مشهود است که توضیحات واضح تر می شوند.



تمرین ۱-۷)

اگر سرعت دو جسم برابر باشد. با توجه به فرمول یعنی f_d آنها نیز برابر خواهد بود. که اینطوری اگر تبدیل فوریه از سیگنال دریافتی بگیریم انگار از یک سینوس که یک فرکانس دارد و فقط دو اختلاف فاز دارد فوریه گرفته ایم. که فوریه آن دلتای دیراک در همان f_d می باشد که سرعت مشترک دو جسم مانند بخش قبل حساب می شود (چون فرکانس را داریم و f_{new} را مانند بخش قبل حساب می کنیم). اما فاز و در نهایت فاصله را نمی توانیم بدست آوریم چون فاز دو سیگنال با هم جمع شده اند و در فوریه نیز قابل تشخیص نیستند.

در زیر نمودار اندازه فوریه سیگنال دریافتی و کد و مقادیر بدست آمده قرار داده شده است.



همانطور که گفتیم چون سرعت ها برابرند یک فرکانس f_d و در نهایت یک فرکانس f_{new} داریم به همین دلیل دو قله یکسان داریم. (قرینه نسبت به محور وای)

```

117 %%
118 R3 = 250*1000;
119 R4 = 200*1000;
120 V3 = 144/3.6;
121 V4 = (144)/3.6;
122
123 td3 = RHO*R3;
124 td4 = RHO*R4;
125 fd3 = BETA*V3;
126 fd4 = BETA*V4;
127
128 y_3 = ALPHA1*cos(2*pi*(fc+fd3)*(t1-td3));
129 y_4 = ALPHA2*cos(2*pi*(fc+fd4)*(t1-td4));
130
131 y_f2 = y_3 + y_4 ;
132 figure;
133 plot(t1,y_f2);
134
135 YF2= fftshift(fft(y_f2));
136 YF2 = YF2/max(abs(YF2));
137 ZF2 = YF2(half_idx);
138 figure;
139 plot(f1, abs(YF2));
140
141 udx = find(ZF2>0.1);
142 f3_new = udx(1);
143 if length(udx)>1
144     f4_new = udx(2);

```

```

145     phi4_new = angle(ZF2(udx(2)));
146
147 else
148     f4_new = udx(1);
149     phi4_new = angle(ZF2(udx(1)));
150 end
151 phi3_new = angle(ZF2(udx(1)));
152 % phi4_new = angle(ZF2(udx(2)));
153
154 fd3_find = f3_new - fc;
155 fd4_find = f4_new - fc;
156 td_find3 = phi3_new / (-2*pi*(fd3_find+fc));
157 td_find4 = phi4_new / (-2*pi*(fd4_find+fc));
158 V3_find = fd3_find / BETA;
159 V4_find = fd4_find / BETA;
160 R3_find = td_find3 / RHO;
161 R4_find = td_find4 / RHO;
162

```

و به همین دلیل به ناچار $phi3_new$ را مساوی $phi4_new$ گرفتیم. و مشخصا فاصله نهایی بدست آمده نیز درست نخواهد بود.

V3	40	8 1x1	R3	250000	8 1x1
V3_find	40	8 1x1	R3_find	2.2273e+...	8 1x1
V4	40	8 1x1	R4	200000	8 1x1
V4_find	40	8 1x1	R4_find	2.2273e+...	8 1x1

حداقل اختلاف سرعت دو جسم چقدر باشد؟

```

N=1*fs;
f1= -fs/2:fs/N:fs/2-fs/N;

```

وقتی به فوریه می‌بریم فرکانس‌های ما صحیح هستند.

```

fs = 100;

```

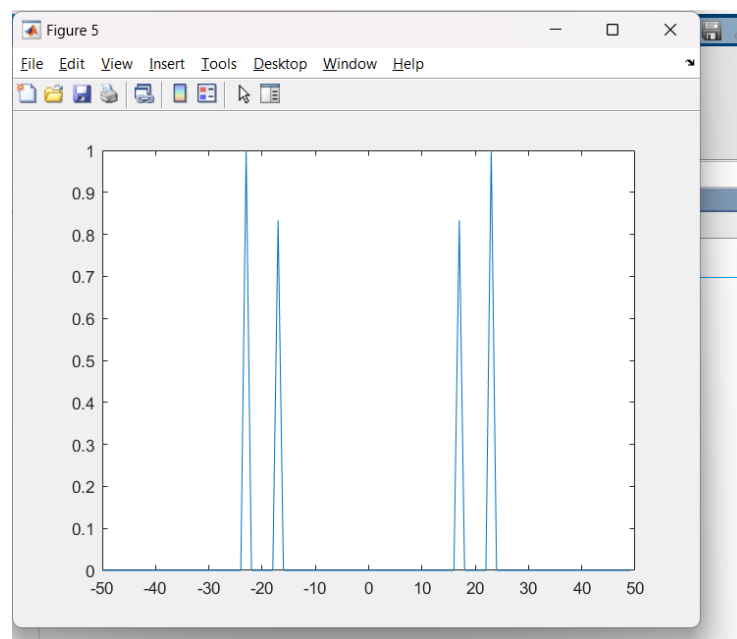
بنابراین باید اختلاف سرعت به گونه‌ای باشد که فرکانس داپلر صحیح بماند که وقتی به حوزه فوریه می‌بریم همان مقدار بماند وگرنه کمی تغییر می‌کند و تمامی متغیرهای خواسته شده ما اشتباه اندازه‌گیری می‌شوند.

$$f_d = \frac{V}{3.6} * \beta \rightarrow f_d \times 12 = V$$

یعنی برای اینکه فرکانس داپلر صحیح باقی بماند باید اختلاف سرعت و خود سرعت ها بر حسب کیلومتر بر ساعت مضرب ۱۲ باشند. (باتوجه به f_s داده شده)

تمرین ۱-۸) به مشکلی ایجاد نخواهد شد. صرفاً t_{delay} ها برابر خواهد داشت که تاثیری روی سرعت نخواهد گذاشت و فاصله ها مجدداً برابر بدست خواهند آمد. (phi_new ها هم برابرند)

تصویر اندازه سیگنال در حوزه فوریه:



```

163 %%
164 R5 = 150*1000;
165 R6 = 150*1000;
166 V5 = 144/3.6;
167 V6 = 216/3.6;
168 ALPHA5 = 0.5;
169 ALPHA6 = 0.6;
170 td5 = RHO*R5;
171 td6 = RHO*R6;
172 fd5 = BETA*V5;
173 fd6 = BETA*V6;
174
175 y_5 = ALPHA1*cos(2*pi*(fc+fd5)*(t1-td5));
176 y_6 = ALPHA2*cos(2*pi*(fc+fd6)*(t1-td6));
177
178 y_f22 = y_5 + y_6 ;
179 figure;
180 plot(t1,y_f22);
181
182 YF22 = fftshift(fft(y_f22));
183 YF22 = YF22/max(abs(YF22));
184 ZF22 = YF22(half_idx);
185 figure;
186 plot(f1, abs(YF22));
187
188 edx = find(ZF22 > 0.1);
189 f5_new = edx(1);
190 if length(edx) > 1
191     f6_new = edx(2);
192     phi6_new = angle(ZF22(edx(2)));
193 else
194     f6_new = edx(1);
195     phi6_new = angle(ZF22(edx(1)));
196 end
197 phi5_new = angle(ZF22(edx(1)));
198
199 fd5_find = f5_new - fc;

```

```

200 fd6_find = f6_new - fc;
201 td_find5 = phi5_new / (-2*pi*(fd5_find + fc));
202 td_find6 = phi6_new / (-2*pi*(fd6_find + fc));
203 V5_find = fd5_find / BETA;
204 V6_find = fd6_find / BETA;
205 R5_find = td_find5 / RHO;
206 R6_find = td_find6 / RHO;

```

کد مربوط به این بخش نیز در بالا قابل رویت می‌باشد.

	V5	40	8 1x1
	V5_find	40	8 1x1
	V6	60	8 1x1
	V6_find	60	8 1x1
	R5	150000	8 1x1
	R5_find	1.5000e+...	8 1x1
	R6	150000	8 1x1
	R6_find	1.5000e+...	8 1x1

درست اندازه‌گیری شده است.

تمرین ۱-۹

در اینجا می‌آییم پس از *fftshift* و جدا کردن از فرکانس صفر تا ۵۰، مقادیر فوریه یافته سیگنال که مقدار بزرگتر از ۰.۱ دارند را در نظر می‌گیریم (چون که بقیه قسمت های فوریه فرکانس به دلیل دلتا فرم بودن تبدیل فوریه سیگنال به دلیل مثلثاتی بودنش مقدار بسیار ناچیزی دارند) و به این ترتیب فرکانس های f_{new} هر کدام پیدا می‌شود و درون آرایه ذخیره می‌کنیم و تعداد اجسام نیز برابر با *length* آرایه مذکور می‌باشد. سایر مراحل مانند بخش قبل است.

```

209 R7 = 250*1000;
210 R8 = 260*1000;
211 R9 = 170*1000;
212 V7 = 144/3.6;
213 V8 = 180/3.6;
214 V9 = 252/3.6;
215 ALPHA3 = 0.4;
216
217 td7 = RHO * R7;
218 td8 = RHO * R8;
219 td9 = RHO * R9;
220
221 fd7 = BETA * V7;
222 fd8 = BETA * V8;
223 fd9 = BETA * V9;
224
225 y_7 = ALPHA1 * cos(2 * pi * (fc + fd7) * (t1 - td7));
226 y_8 = ALPHA2 * cos(2 * pi * (fc + fd8) * (t1 - td8));
227 y_9 = ALPHA3 * cos(2 * pi * (fc + fd9) * (t1 - td9));
228
229 y_fn = y_7 + y_8 + y_9;
230
231
232 YFN = fftshift(fft(y_fn));
233 YFN = YFN / max(abs(YFN));
234 ZFN = YFN(half_idx);
235
236 pdx = find(ZFN > 0.1);
237 n = length(pdx);
238
239
240 f_n_new = zeros(1, n);
241 phi_n_new = zeros(1, n);
242 fd_find = zeros(1, n);
243

```

```

239
240 f_n_new = zeros(1, n);
241 phi_n_new = zeros(1, n);
242 fd_find = zeros(1, n);
243 td_find = zeros(1, n);
244 V_find = zeros(1, n);
245 R_find = zeros(1, n);
246
247 for i = 1:n
248     f_n_new(i) = pdx(i);
249     phi_n_new(i) = angle(ZFN(pdx(i)));
250     fd_find(i) = f_n_new(i) - fc;
251     td_find(i) = phi_n_new(i) / (-2*pi*(fd_find(i) + fc));
252     V_find(i) = fd_find(i) / BETA;
253     R_find(i) = td_find(i) / RHO;
254 end
255
256 disp('Frequencies found:');
257 disp(fd_find);
258 disp('Velocities found:');
259 disp(V_find);
260 disp('Ranges found :');
261 disp(R_find);
262 disp('t_delays found :');
263 disp(td_find);
264

```

خروجی در زیر هم t_{delay} ها را می نویسد.

: Frequencies found:

12 15 21

Velocities found:

40 50 70

Ranges found :

1.0e+05 *

2.5000 2.6000 1.7000

t_delays found :

0.0017 0.0017 0.0011

RG	RG	RG
R7	250000	8 1x1
R8	260000	8 1x1
R9	170000	8 1x1
V7	40	
V8	50	
V9	70	

که درست مشخص شده است.

بخش دوم)

FILE P2.m

تمرین ۲-۱)

1	clear all;	
2	clc;	
3	%%	
4	T = 0.5;	
5	t_total = 8*4*T;	
6	t_total = t_total + 43*0.025;	
7	fs = 8000;	
8		
9	D=587.33;	
10	E=659.25;	
11	Fsharp = 739.99;	
12	G=783.99;	
13		
14	dlytime = 0.025;	
15		
16	%%%%%%%%%	
17	t = 0:1/fs:t_total - 1/fs;	
18	y = ones(size(t));	
19		
20		

مقادیر اولیه و تولید آرایه به صورت استاتیک (یعنی از اول کل دیلی ها و زمان نوت هارو حساب کردم و آرایه با اون سایز مخصوص به آهنگ داده شده رو ساختم و بعدش میام ایندکس تا ایندکس مقادیر سیگنال y رو تغییر میدم و نوت پیانو سینوسی یا دیلی ک همون $\cdot$ هست رو ذخیره میکنم داخل آرایه)

برای اینکار دو تابع ایجاد کردم.

436	%%	
437		
438	function dly = delay()	
439	fs=8000;	
440	t = 0:1/fs:0.025-1/fs ;	
441	dly = zeros(size(t));	
442	%%%	
443	end	
444		
445	function note = piano_note(fc , duration)	
446	fs=8000;	
447	t= 0:1/fs:duration-1/fs ;	
448	x = sin(2*pi*fc*t);	
449	note = x;	
450		
451	end	

تابع های دیلی و نت که به ترتیب برای زمانی توقف بین دو کلید و مونو پیانو زدن (تک کلید) هست.

که پیانو زدن میاد سیگنال سینوسی با فرکانس اون نت رو در ایندکس های مربوط به زمان خودش ذخیره می کنه.

دیلی که نیازی به ورودی خاصی نداره. ورودی تابع نوت هم فرکانس اون کلید پیانو و مدت زمانی که اون کلید قراره پخش بشه هست $T/2$ یا T .

باقی کار ساده هست.

از دو متغیر `current_end_time` , `current_begin_time` برای مقدار دهی درون آرایه استفاده کردم.

```

22
23     current_end_time = T/2;
24     y(1:current_begin_time*fs : current_end_time*fs) = piano_note(D,T/2);
25     current_begin_time = current_end_time;
26
27     current_end_time=current_end_time+dlytime;
28     y( current_begin_time*fs+1:current_end_time*fs )= delay();
29     current_begin_time=current_end_time;
30     %%%
31     current_end_time = current_end_time+T/2;
32     y( current_begin_time*fs+1:current_end_time*fs ) = piano_note(D,T/2);
33     current_begin_time = current_end_time;
34
35     current_end_time=current_end_time+dlytime;
36     y( current_begin_time*fs+1:current_end_time*fs )= delay();
37     current_begin_time=current_end_time;
38     %%%
39     current_end_time = current_end_time+T;
40     y( current_begin_time*fs+1:current_end_time*fs ) = piano_note(G,T);
41     current_begin_time = current_end_time;
42
43     current_end_time=current_end_time+dlytime;
44     y( current_begin_time*fs+1:current_end_time*fs )= delay();
45     current_begin_time=current_end_time;
46     %%%
47     current_end_time = current_end_time+T;
48     y( current_begin_time*fs+1:current_end_time*fs ) = piano_note(Fsharp,T);
49     current_begin_time = current_end_time;
50

```

به ترتیب نت ها و دیلی ها رو یک درمیون گذاشتم و در γ آهنگ رو ذخیره کردم.

```

384
385
386     %%%
387
388     sound(y, fs);
389     %%%%%%%%%%
390     %%

```

و با این دستور هم آهنگ ساخته شده که احتمالا اولین دقیقه و سومین ثانیه پیانو آهنگ *love story* از تیلور سویفت هست پلی میشه.

تمرین ۲-۲)

FILE P2_1.m

این دفعه آرایه رو داینامیک ساختم . چون مقدار دهی و طولانی بودن خط ها به صرفه نبود . دو تابع قبلی که تکراری هستند. فقط این دفعه برای مقداردهی آهنگ به صورت دینامیک

$y = [y_{new_part_of_the_song}]$

اینطوری اضافه کردم. و نت ها و زمان اجراشون رو هم درون آرایه ذخیره کردم و با حلقه فور به آهنگ اضافه می کنه و فراموش هم نمیکنه بعد هر نت پیانو باید یک دیلی بذاره.

آهنگی که انتخاب شده هم

```
clear all;
clc;

T = 0.55;
fs = 8000;
dlytime = 0.025;

C = 523.25;
Cs = 554.37;
D = 587.33;
Ds = 622.25;
E = 659.25;
F = 698.46;
Fs = 739.99;
G = 783.99;
Gs = 830.61;
A = 880;
As = 932.33;
B = 987.77;

melody = [
    C T; C T; G T; G T; A T; A T; G 2*T;
    F T; F T; E T; E T; D T; D T; C 2*T;
    G T; G T; F T; F T; E T; E T; D 2*T;
    G T; G T; F T; F T; E T; E T; D 2*T;
    C T; C T; G T; G T; A T; A T; G 2*T;
    F T; F T; E T; E T; D T; D T; C 2*T;
];

y = [];
for i = 1:size(melody, 1)
    freq = melody(i,1);
    dur = melody(i,2);
    y = [y piano_note(freq, dur) delay(dlytime)];
end
```

```

y = [];
for i = 1:size(melody, 1)
    freq = melody(i,1);
    dur = melody(i,2);
    y = [y piano_note(freq, dur) delay(dlytime)];
end

sound(y, fs);

filename = 'twinkle_twinkle.wav';
audiowrite(filename, y, fs);

function dly = delay(dlytime)
    fs=8000;
    t = 0:1/fs:dlytime-1/fs;
    dly = zeros(size(t));
end

function note = piano_note(fc , duration )
    fs=8000;
    t= 0:1/fs:duration-1/fs ;
    note = sin(2*pi*fc*t);
end

```

در اواخر کد هم با دستور اودیو رایت همونطور که گفته شده در پوشه اصلی پروژه اهنگ ساخته شده ذخیره شد.

تمرین ۲-۳

File p2.m from line 390

$$x(t) = \sin(2\pi ft) \cdot \text{rect}\left(\frac{t - t_1}{\alpha}\right)$$

سیگنال موقع پیانو زدن به شکل بالاست.

و در زمان دیلی هم مقدارش صفره ک فوریه اش صفر هست.

$$X(f) = \frac{\alpha}{2j} \left[\text{sinc}(\alpha(f - f_0)) \cdot e^{-j2\pi(f-f_0)t_1} - \text{sinc}(\alpha(f + f_0)) \cdot e^{-j2\pi(f+f_0)t_1} \right]$$

و این تبدیل فوریه اش هست که همونطور که مشاهده میشه به صورت یک سینک شیفته یافته هست.

ماکسیمم مقدار سینک کجاست؟ در همون چیزی که ضابطه اش رو صفر می کنه. یعنی نوت پیانو مثل دفعات قبل فرکانسش با همون فرکانس ماکزیمم تبدیل فوریه برابره.

و مشکل دوم ما چیه؟ این تابع مستطیل که داریم طولش یک نیست بلکه به اندازه دوره تناوب هست! یعنی وقتی میبریمش توی فوریه همون فرکانس ک صفر میشه فرکانس نوت پیانو ما نیست بلکه یک عددی ضرب شده داخلش که همون ضربی از دوره تناوب هست... بستگی به مدت زمان نگه داشتن کلیدش داره....

الگوریتم من چی بود برای حل این سوال که با ناقص موند با توجه به اتمام زمان آپلود:

۴ تا نوت استفاده شده در اهنگ بخش ۱ رو میریزم داخل آرایه ای به نام نوتز

یک آرایه دیگه به نام تایمز میسازم و المنت های هم ایندکس آرایه نوتز میشن زمان کلی پلی شدن اون نوت در پیانو.

بعد میام داخل سیگنال در حوزه زمان

کل نقطه هارو بررسی میکنم.

هرجا همون نقطه i و نقطه $i-1$ اش صفر و نقطه $i+1$ اش نامساوی با صفر باشه (یعنی بعد از دیلی و اول شروع پیانو زدن باشه) رو المنتش رو مینویسیم t_begin

هرجا همون نقطه i و نقطه $i+1$ اش صفر و نقطه $i-1$ اش نامساوی با صفر باشه (یعنی بعد از دیلی و اول شروع پیانو زدن باشه) رو المنتش رو مینویسیم t_end

و میام

$t_end - t_begin$ می کنم و ضربدر ۱ تقسیم بر ۸۰۰۰ که همون fs بود می کنم و زمان پلی شدن اون نوت خاص رو بهم می گه.

بعد میام از المنت t_begin تا t_end رو میرم در حوزه فوریه و اونجا قله و ایندکس مربوطش ک همون ضریبی از فرکانس نوت خاص پیانوم رو هست پیدا می کنم . تقسیم بر اون ضریبه می کنیم و فرکانسش رو پیدا می کنم . و وقتی فرکانسش رو پیدا کردم . میام میبینم داخل آرایه نوتز کدوم المنت با چه ایندکسی با اون فرکانسه برابره.... میام در آرایه تایمز اون ایندکس رو پیدا می کنیم و مقدار قبلش رو به اضافه ی مقدار پلی شدن جدید نوت پیانو می کنم.

که در پیدا کردن اون ضریبه که من رو به فرکانس نوت پیانوم میرسونه ناموفق بودم (احتمالا ضربدر یک ضریبی از دوره تناوب باید باشه)

و در آخر میام و میگم که کدوم نوت ها هرکدوم چند ثانیه اجرا شدن.

کد مربوط به توضیحات بالا نیز در زیر قرار داده شده است.

```

%%
yol = length(y);
notes = [ (D) (E) (Fsharp) (G)];
times = [0 0 0 0];
begin_time=1;
end_time=0;
flag=0;
for i = 2:yol-1
    % fprintf("%d\n",i);
    if i==yol-1
        flag=1;
    end
    if ((y(i)==0 && y(i-1)~=0 && y(i+1) ==0)||flag==1)
        if flag==1
            i=yol+1;
        end
        end_time = i;
        timeplay = (end_time-begin_time)/8000;
        if flag==1
            end_time = end_time-1;
        end
        z = (y(begin_time:end_time));
        Z = fftshift(fft(z));
        ZS = length(Z);
        half_idx = (floor(ZS/2)+2):ZS;
        U = Z; %(half_idx);
        [maxVal , jdx] = max(U);
        fprintf("%d %d\n",timeplay,jdx);
        for k =1:4
            if round(jdx)==round(notes(k))
                times(k)=times(k)+timeplay;
            end
        end
    end
end

```

```

418         for k =1:4
419             if round(jdx)==round(notes(k))
420                 times(k)=times(k)+timeplay;
421             end
422         end
423
424         %         flag=0;
425     end
426     if flag==1
427         i=i-1;
428     end
429     if y(i)==0 && y(i+1)~=0 && y(i-1)==0
430
431         begin_time = i;
432
433     end
434
435 end

```

دو عکس زیر هم به ترتیب دیلی ها و فرکانس های ماکزیمم در حوزه فوریه رو نشون می دن چون ناتموم موند این بخش از کد. این قسمت قرار داده شده است.

```
2.500000e-01 854
2.500000e-01 854
5.000000e-01 1609
5.000000e-01 1631
5.000000e-01 1707
2.500000e-01 854
2.500000e-01 836
2.500000e-01 836
2.500000e-01 854
2.500000e-01 816
2.500000e-01 854
5.000000e-01 1671
5.000000e-01 1707
5.000000e-01 1671
5.000000e-01 1631
5.000000e-01 1671
2.500000e-01 854
2.500000e-01 836
2.500000e-01 836
2.500000e-01 854
2.500000e-01 816
2.500000e-01 854
5.000000e-01 1671
5.000000e-01 1707
fx 2.500000e-01 836
```

fx

||||

```
2.500000e-01 854
5.000000e-01 1631
5.000000e-01 1671
5.000000e-01 1707
2.500000e-01 836
2.500000e-01 854
5.000000e-01 1631
5.000000e-01 1671
2.500000e-01 854
2.500000e-01 854
5.000000e-01 1671
2.500000e-01 816
2.500000e-01 836
5.000000e-01 1631
2.500000e-01 816
2.500000e-01 836
5.000000e-01 1631
5.000000e-01 1631
5.000000e-01 1707
```

fx >>

||||

